

LAM7ARA

Sistema de Gestión de Inventario y Ventas

Aplicación de escritorio para punto de venta

SEGUNDA ENTREGA

Autor:	Bruno Depetris
--------	----------------

Docente Tutor:	Gonzalo Juan Castillo
----------------	-----------------------

Ciclo Lectivo:	2026
----------------	------

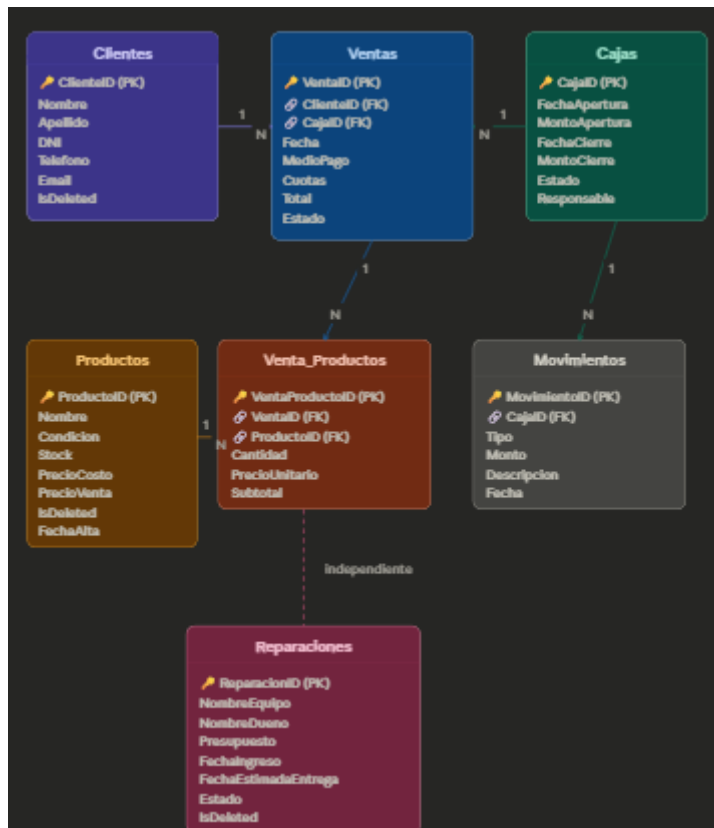
Esquema de Base de Datos

Se presentan los dos diagramas UML de Entidad-relación (DER) que representan la estructura de datos del sistema LAM7ARA. El primero muestra la visión conceptual del dominio y el segundo la implementación física en SQLite.

DER Conceptual

El diagrama conceptual representa las entidades principales del dominio, sus atributos clave y las relaciones entre ellas, sin incluir detalles técnicos de implementación.

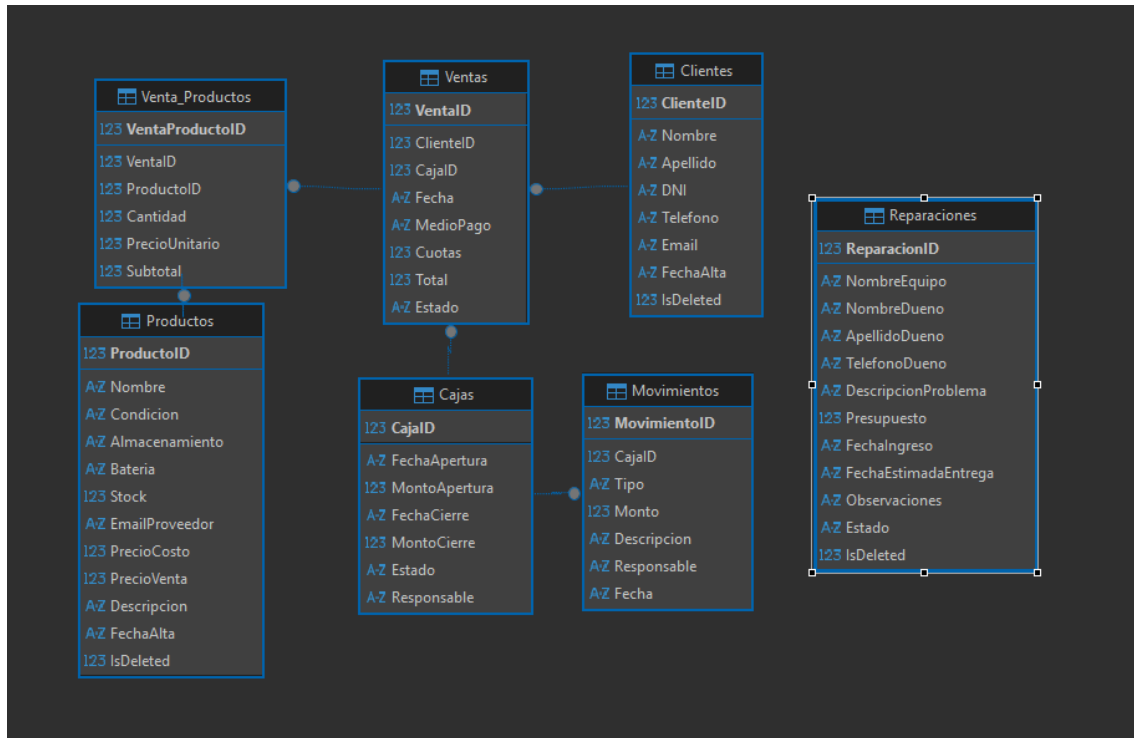
Entidades principales: Clientes, Productos, Ventas, Venta_Productos, Cajas y Movimientos. Las relaciones principales son: un Cliente puede tener muchas Ventas; una Venta contiene uno o más Productos a través de la tabla intermedia Venta_Productos; una Venta pertenece a una Caja; una Caja puede tener muchos Movimientos.



El

DER físico

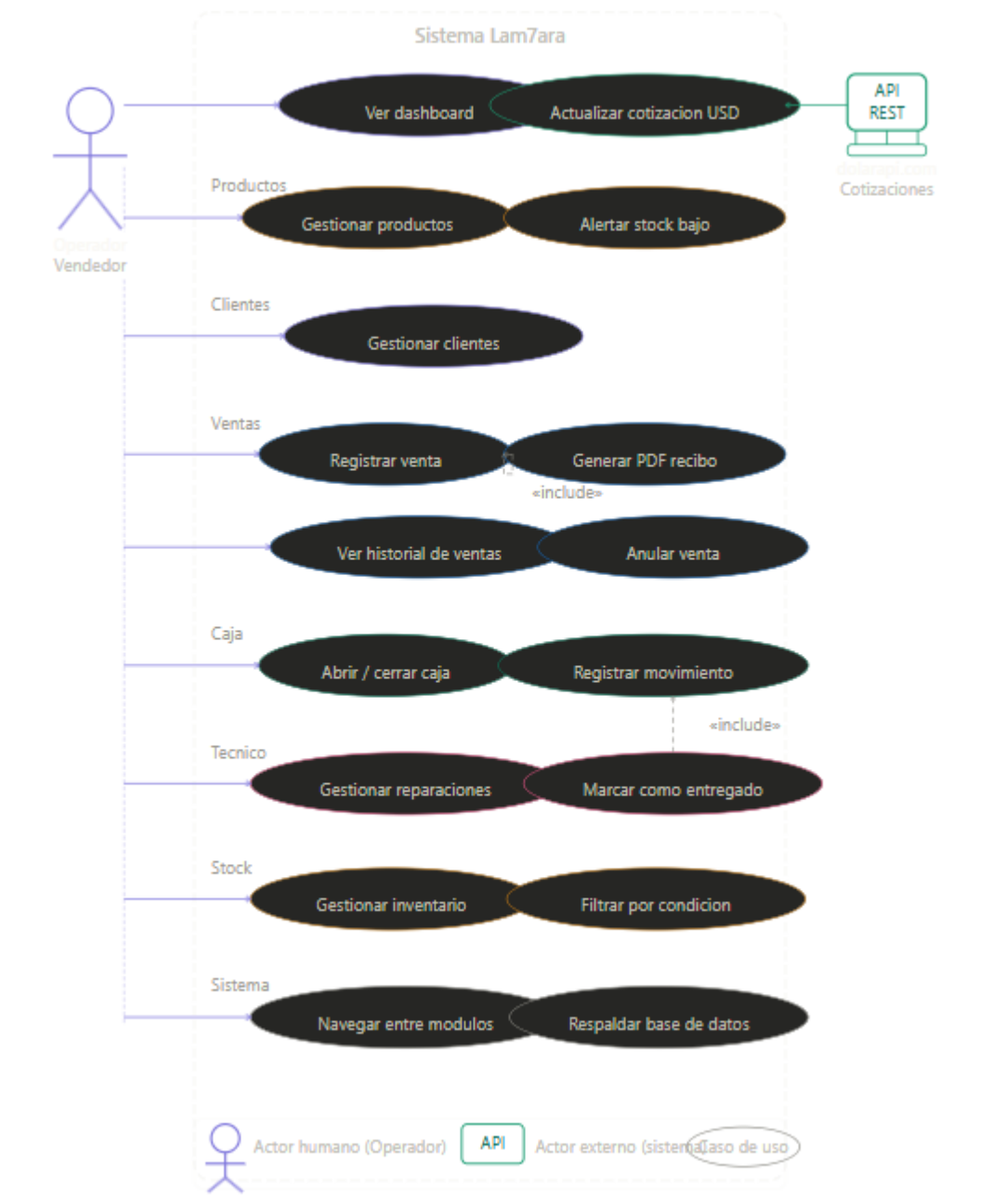
El diagrama físico representa la implementación exacta de la base de datos en SQLite, incluyendo claves primarias (PK), claves foráneas (FK), tipos de datos y restricciones CHECK. La base de datos se almacena en el archivo Lam7araDataBaseAPP.db ubicado en %AppData%\Lam7ara\ y se activa PRAGMA foreign_keys = ON al iniciar.



Listado de módulos a desarrollar

El sistema LAM7ARA cuenta con un único tipo de actor principal (Operador/Vendedor) ya que no implementa sistema de autenticación ni roles diferenciados. Adicionalmente se identifica un actor externo: la API REST de dolarapi.com que provee cotizaciones de divisas.

A continuación se presenta el diagrama UML de Casos de Uso del sistema completo y el detalle de cada módulo con sus funcionalidades, pantallas e interfaces relacionadas.



Detalle de Módulos

Módulo 1: Gestión de Productos

Objetivo: Permitir la administración completa del catalogo de productos del sistema, incluyendo alta, baja, modificación y consulta de productos con sus especificaciones técnicas.

Usuarios: Operador / Vendedor

Pantallas: FormStock: lista de productos con filtro por condición (Nuevo/Usado), búsqueda por texto, resaltado de stock bajo (≤ 3 unidades), y panel lateral de edición.

Interfaces relacionadas: Base de datos SQLite (tabla Productos). Servicio: ProductoService con métodos Listar(), BuscarPorID(), Buscar(), Agregar(), Editar(), Eliminar() (baja logica via IsDeleted) y ActualizarStock() (usado internamente por VentaService en transacción).

Módulo 2: Gestión de Clientes

Objetivo: Administrar los registros de clientes del comercio, permitiendo el alta, modificación, eliminación lógica y búsqueda de clientes.

Usuarios: Operador / Vendedor

Pantallas: FormCLIENTES: grilla de clientes con búsqueda por nombre, apellido o DNI, y panel lateral de edición. Modo Nuevo / Modo edición con etiqueta dinámica.

Interfaces relacionadas: Base de datos SQLite (tabla Clientes). Servicio: ClienteService con métodos Listar(), BuscarPorID(), Buscar(), Agregar(), Editar(), Eliminar().

Módulo 3: Gestión de Ventas

Objetivo: Registrar transacciones de venta vinculando múltiples productos, clientes y medios de pago, con emisión de comprobantes PDF y gestiona del historial.

Usuarios: Operador / Vendedor

Pantallas: FormVender: búsqueda de cliente, búsqueda y agregado de productos a grilla, selección de medio de pago (Efectivo/Transferencia/crédito), selección de cuotas (1/3/6/12/18/24) si es crédito, total acumulado y botones Concretar/Cancelar. FormVerVentas: historial filtrable por fecha y medio de pago, detalle de productos por venta, total, anulación y generación de PDF.

Interfaces relacionadas: Tablas SQLite: Ventas, Venta_Productos, Clientes, Productos, Cajas. Servicios: VentaService, ClienteService, ProductoService,

CajaService. FormSelectorCliente y FormSelectorProducto como diálogos de selección cuando la búsqueda retorna múltiples resultados. iTextSharp para generación de PDF del comprobante.

Módulo 4: Caja Registradora

Objetivo: Gestionar las operaciones diarias de caja, incluyendo apertura con monto inicial, registro de movimientos (ingresos/egresos) y cierre con saldo calculado automáticamente.

Usuarios: Operador / Vendedor

Pantallas: FormMovimientos: panel de cards de estado (estado caja, monto apertura, saldo actual, ventas del día), acciones de apertura/cierre, grilla de movimientos de la caja activa con colores por tipo, formulario de nuevo movimiento, e historial de cajas anteriores.

Interfaces relacionadas: Tablas SQLite: Cajas, Movimientos, Ventas. Servicios: CajaService, MovimientoService. Saldo calculado = MontoApertura + Ingresos - Egresos + TotalVentas.

Módulo 5: Dashboard (Panel de Control)

Objetivo: Proporcionar una vista consolidada del estado del negocio con indicadores clave, cotización del dólar en tiempo real y alertas de stock bajo.

Usuarios: Operador / Vendedor

Pantallas: FormInicio: 4 cards de métricas (ventas del día \$, cantidad de ventas, clientes registrados, estado de caja), 3 cards de cotizaciones USD (oficial, blue, tarjeta) con botón Actualizar, grilla de productos con stock bajo y tabla de ultimas 10 ventas.

Interfaces relacionadas: Consultas directas SQLite sobre Ventas, Clientes, Productos, Cajas, Movimientos. API REST dolarapi.com (GET /v1/dolares) con ApiCotizacion (HttpClient) y deserializacion via Newtonsoft.Json en ModeloApiCotizacion. Las cotizaciones se actualizan manualmente con el botón.

Módulo 6: Navegación Principal

Objetivo: Proveer un hub centralizado de navegación con sidebar moderno que permite acceder a todos los módulos del sistema.

Usuarios: Operador / Vendedor

Pantallas: FormMAIN: sidebar dark con header de branding (logo + nombre + subtítulo), botones de navegación con indicador activo de 4px, reloj en tiempo real

en footer del sidebar, topbar con título dinámico y botones de minimizar/cerrar. panelMainContent como contenedor dinámico de módulos.

Interfaces relacionadas: Timer de reloj (Interval=1000ms). Cada botón de navegación instancia el formulario correspondiente, establece TopLevel=false, Dock=Fill y lo agrega al panelMainContent.